

Exercício Prático: API de Gestão de Estudantes

Turma: LionsDev **Tópicos:** APIs REST com Express.js, JSON, CRUD, parâmetros de rota, query params, validação de dados e status codes HTTP.

1. Contexto

A secretaria de uma escola precisa de uma API simples para cadastrar, listar, atualizar, remover e buscar estudantes. O sistema ainda não usará banco de dados real: todos os dados ficarão armazenados em um array de objetos enquanto o servidor estiver rodando.

O objetivo deste exercício é transformar operações de CRUD em rotas HTTP usando o Express.

2. Configuração Inicial

Crie uma pasta para o projeto e configure uma API Node.js com Express.

Requisitos iniciais:

- O arquivo principal deve se chamar `index.js`.
- A API deve rodar na porta `3000`.
- Use `express.json()` para permitir o recebimento de dados em JSON.
- Crie um array chamado `estudantes` para armazenar os registros.
- Crie uma variável `proximoId` iniciando em `1` para gerar IDs automáticos.

Cada estudante deve seguir esta estrutura:

```
{
  id: 1,
  nome: "Ana Souza",
  matricula: "2026001",
  curso: "Desenvolvimento Web",
  ano: 2026
}
```

3. Requisitos Obrigatórios

3.1 Criar estudante (CREATE)

Crie a rota **POST /estudantes/criar** .

O corpo da requisição deve receber:

```
{
  "nome": "Ana Souza",
  "matricula": "2026001",
  "curso": "Desenvolvimento Web",
  "ano": 2026
}
```

Regras:

- Todos os campos (**nome** , **matricula** , **curso** , **ano**) são obrigatórios.
- Se algum campo estiver faltando, responda com status **400** e uma mensagem de erro.
- O **id** deve ser gerado automaticamente usando **proximoId** .
- Ao cadastrar com sucesso, responda com status **201** e retorne o estudante criado.

3.2 Listar estudantes (READ)

Crie a rota **GET /estudantes** .

Regras:

- A rota deve retornar todos os estudantes cadastrados.
- Se não houver estudantes, retorne um array vazio **[]** .
- Use status **200** para a resposta.

3.3 Atualizar estudante (UPDATE)

Crie a rota **PUT /estudantes/:id** .

O corpo da requisição pode receber um ou mais campos para alteração:

```
{
  "curso": "Backend com Node.js",
  "ano": 2027
}
```

Regras:

- Busque o estudante pelo **id** recebido em **req.params** .
- Converta o **id** para número antes de comparar.
- Se o estudante não existir, responda com status **404** e uma mensagem de erro.

- Atualize apenas os campos enviados no corpo da requisição.
- Retorne o estudante atualizado com status **200**.

3.4 Deletar estudante (DELETE)

Crie a rota **DELETE /estudantes/:id**.

Regras:

- Busque o índice do estudante usando **.findIndex()**.
- Se o estudante não existir, responda com status **404** e uma mensagem de erro.
- Remova o estudante do array usando **.splice()**.
- Retorne uma mensagem de sucesso com status **200**.

3.5 Buscar estudantes (QUERY PARAMS)

Crie a rota **GET /estudantes/busca**.

A busca deve aceitar os seguintes filtros opcionais:

- **nome**
- **matricula**
- **curso**

Exemplos de URLs:

```
http://localhost:3000/estudantes/busca?nome=ana
http://localhost:3000/estudantes/busca?curso=web
http://localhost:3000/estudantes/busca?matricula=2026
```

Regras:

- A busca por **nome** deve aceitar parte do texto e ignorar letras maiúsculas/minúsculas.
- A busca por **curso** também deve aceitar parte do texto e ignorar letras maiúsculas/minúsculas.
- A busca por **matricula** deve aceitar parte da matrícula digitada.
- Se mais de um filtro for enviado, aplique todos os filtros em sequência.
- Retorne os resultados encontrados com status **200**.

4. Testes Esperados

Teste sua API no Postman.

Sugestões de testes:

1. Cadastre pelo menos 3 estudantes.
2. Liste todos os estudantes.
3. Atualize apenas o curso de um estudante.
4. Tente atualizar um `id` que não existe.
5. Delete um estudante existente.
6. Tente deletar um `id` que não existe.
7. Busque estudantes por `nome`, `matricula` e `curso`.

5. Dicas para a Implementação

- `req.body` guarda os dados enviados no corpo da requisição.
- `req.params` guarda valores enviados na rota, como o `id` em `/estudantes/:id`.
- `req.query` guarda filtros enviados depois do `?` na URL.
- Use `parseInt()` para converter o `id` recebido pela rota.
- Para buscas por texto, use `.toLowerCase()` junto com `.includes()`.
- Para atualizar apenas campos enviados, você pode usar a regra do `valorNovo || valorAntigo`.

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de APIs REST com Express.js - Módulo 07